Respuestas - Ejercicios Econometría Taller 1

Introducción a la computación estadística y econometría

1. Nombre y apellido alternados

Se utilizan Ana y Perez como ejemplo. Diez iteraciones muestran cinco veces cada texto, de manera alternada. Reemplaza estos valores por tu nombre y tu apellido.

Con for

```
nombre <- "Ana"
apellido <- "Perez"
for (i in 1:10) {
  if (i %% 2 == 1) {
    print(nombre)
  } else {
    print(apellido)
  }
}
```

Con while

```
i <- 1
while (i <= 10) {
  if (i %% 2 == 1) {
    print(nombre)
  } else {
    print(apellido)
  }
  i <- i + 1
}
```

El resto de dividir `i` por 2 se obtiene con `i %% 2`: es 1 en las posiciones impares y 0 en las pares. Por eso se imprime primero el nombre y después el apellido. En el `while`, aumentar `i` permite llegar al final.

2. Sucesión hasta -20

Hay 40 posiciones: cada número de 1 a 20 aparece una vez positivo y una vez negativo. `ceiling(i / 2)` redondea hacia arriba y genera 1, 1, 2, 2, etc.; la condición elige el signo.

Con for

```
sucesion_for <- numeric(40)
for (i in 1:40) {
  numero <- ceiling(i / 2)
  if (i %% 2 == 1) {
    sucesion_for[i] <- numero
  } else {
    sucesion_for[i] <- -numero
  }
}
print(sucesion_for)
```

Con while

```
sucesion_while <- numeric(40)
i <- 1
while (i <= 40) {
  numero <- ceiling(i / 2)
  if (i %% 2 == 1) {
    sucesion_while[i] <- numero
  } else {
    sucesion_while[i] <- -numero
  }
  i <- i + 1
}
print(sucesion_while)
```

Ambos resultados comienzan con $1, -1, 2, -2$ y terminan con $19, -19, 20, -20$.

3. Funciones para construir vectores y secuencias

Función	Uso y resultado del ejemplo
`c()`	Reúne valores en un vector. Por ejemplo: `c(2, 4, 6)` produce 2, 4, 6.
`seq()`	Construye una secuencia. Por ejemplo, para construir un vector de 2 a 10, con salto 2: `seq(2, 10, by = 2)` produce 2, 4, 6, 8, 10.
`numeric()`	Crea un vector de ceros con la longitud indicada. Por ejemplo: `numeric(4)` produce 0, 0, 0, 0.
`rep()`	Repite valores. Por ejemplo: `rep(c("A", "B"), each = 2)` produce A, A, B, B.
`seq_len()`	Genera posiciones desde 1 hasta la longitud indicada. Por ejemplo: `seq_len(4)` produce 1, 2, 3, 4. Con `seq_len(0)` se obtiene una secuencia vacía.
`seq_along()`	Genera las posiciones de un vector existente. Por ejemplo: `seq_along(c(10, 20, 30))` produce 1, 2, 3.
`sequence()`	Une secuencias cuyas longitudes se indican. Por ejemplo: `sequence(c(2, 3))` produce 1, 2, 1, 2, 3.
`gl()`	Genera grupos llamados niveles. Por ejemplo: `gl(2, 3, labels = c("A", "B"))` produce A, A, A, B, B, B, con los niveles A y B.

```
c(2, 4, 6)
seq(from = 2, to = 10, by = 2)
numeric(4)
rep(c("A", "B"), each = 2)
seq_len(4)
v_ejemplo <- c(10, 20, 30)
seq_along(v_ejemplo)
sequence(c(2, 3))
gl(2, 3, labels = c("A", "B"))
```

En `gl(2, 3, labels = c("A", "B"))`, 2 es la cantidad de grupos, 3 es la cantidad de repeticiones consecutivas por grupo y `labels` asigna sus nombres. El resultado es un **factor**: una forma de guardar categorías.

4. Números aleatorios y floor

`runif(n, min, max)` utiliza `n` para la cantidad de valores y `min` y `max` para los límites.

£Para qué se fija una semilla? R genera números seudoaleatorios: parecen aleatorios, pero se obtienen mediante un procedimiento. La semilla fija el punto de partida de ese procedimiento. Si vuelves a ejecutar `set.seed(123)` y después las mismas instrucciones aleatorias, en el mismo orden y con el mismo método de generación, obtendrás los mismos valores.

£Por qué 123? Es una elección arbitraria, fácil de recordar; no tiene una propiedad especial ni hace que la muestra sea mejor. Puedes usar otro entero válido, como 42 o 2026. Normalmente cambiará la secuencia de valores, por eso es importante dejar escrito el número que utilizaste.

£Qué pasa si pierdo la semilla? Si no registraste la semilla ni conservaste el estado del generador, no puedes garantizar que volverás a obtener exactamente la misma muestra al ejecutar de nuevo el código. Si guardaste los datos generados o las filas seleccionadas, puedes reutilizarlos sin volver a sortearlos. Conservar la semilla en el código permite repetir el procedimiento, siempre que también se mantengan sus datos, instrucciones y método de generación.

Fija la semilla una sola vez antes de generar los números o antes de iniciar un ciclo de muestreo. Si la reinicias con el mismo número dentro de cada iteración, repetirás la misma selección en lugar de obtener nuevas muestras.

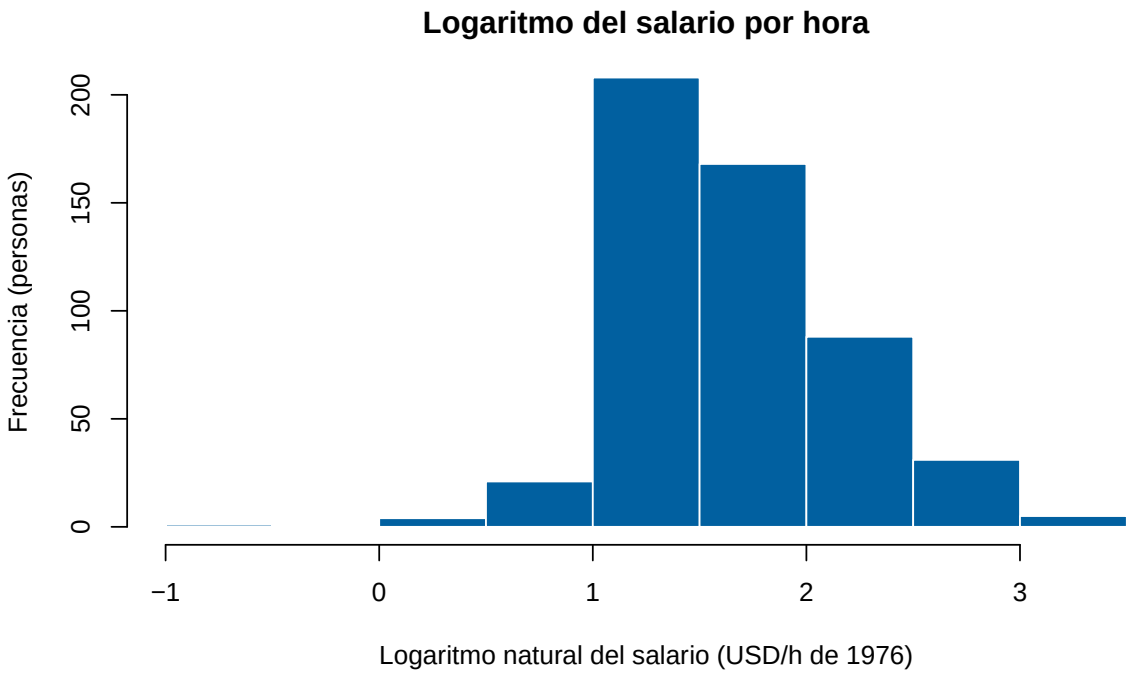
```
set.seed(123)
v <- runif(n = 1000, min = 0, max = 1)
v_enteros <- floor(v * 100)
head(data.frame(v = v, multiplicado = v * 100,
                resultado = v_enteros))
length(v)
range(v_enteros)
```

Con esta semilla, los primeros seis resultados transformados son **28, 78, 40, 88, 94 y 4**. Al multiplicar por 100, los valores quedan entre 0 y 100, sin llegar a 100. `floor()` devuelve el mayor entero menor o igual que cada valor: por ejemplo, transforma 28,75775 en 28. El resultado toma valores de **0 a 99**. Son valores enteros, aunque R los guarda aquí como números de tipo `numeric`.

5. Histograma del logaritmo del salario

```
# Instalar solo si el paquete no esta instalado:
# install.packages("wooldridge", repos = "https://cloud.r-project.org/")
datos <- wooldridge::wage1
```

```
y <- datos$wage
hist(log(y), main = "Logaritmo del salario por hora",
     xlab = "Logaritmo natural del salario (USD/h de 1976)",
     ylab = "Frecuencia (personas)",
     col = "#0160a0", border = "white")
```



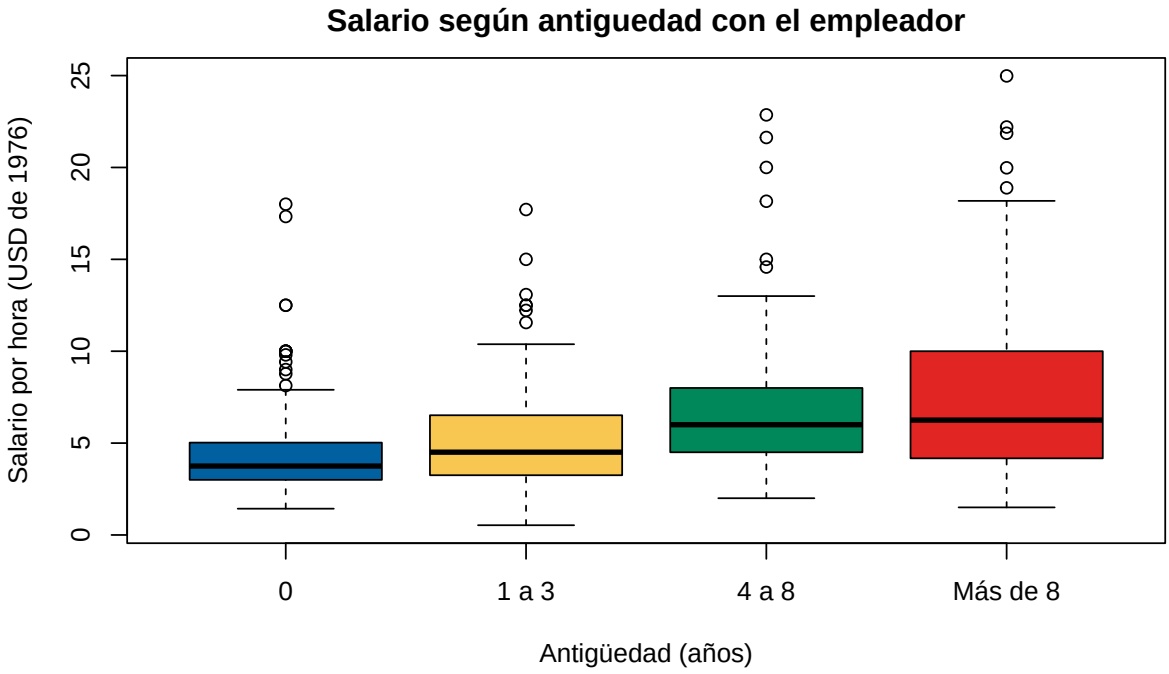
Descripción de lo observado. El histograma concentra la mayoría de los valores del logaritmo del salario entre 1 y 2, y presenta una cola hacia valores más altos. Hay pocas observaciones en los extremos. El eje vertical cuenta personas; el horizontal muestra el logaritmo natural del salario por hora, no el salario original.

Un valor negativo en el eje horizontal no significa un salario negativo: significa que el salario original es positivo y menor que 1 USD por hora. La transformación comprime las diferencias entre salarios altos. En esta base, la media salarial original es aproximadamente **5,896 USD/h** y su mediana es **4,650 USD/h**; no deben confundirse con los valores de la escala logarítmica.

Datos: 526 personas de la CPS de Estados Unidos, 1976. Para verificar las respuestas se utilizó la copia local de wage1, sin cambiar sus valores.

6. Salarios según antigüedad

```
datos$tenure_grupo <- cut(
  datos$tenure, breaks = c(-Inf, 0, 3, 8, Inf),
  labels = c("0", "1 a 3", "4 a 8", "Más de 8"),
  right = TRUE
)
table(datos$tenure_grupo)
boxplot(wage ~ tenure_grupo, data = datos,
        col = c("#0160a0", "#f8c751", "#00875a", "#e12523"),
        main = "Salario según antiguedad con el empleador",
        xlab = "Antigüedad (años)",
        ylab = "Salario por hora (USD de 1976)")
aggregate(wage ~ tenure_grupo, data = datos, FUN = median)
```



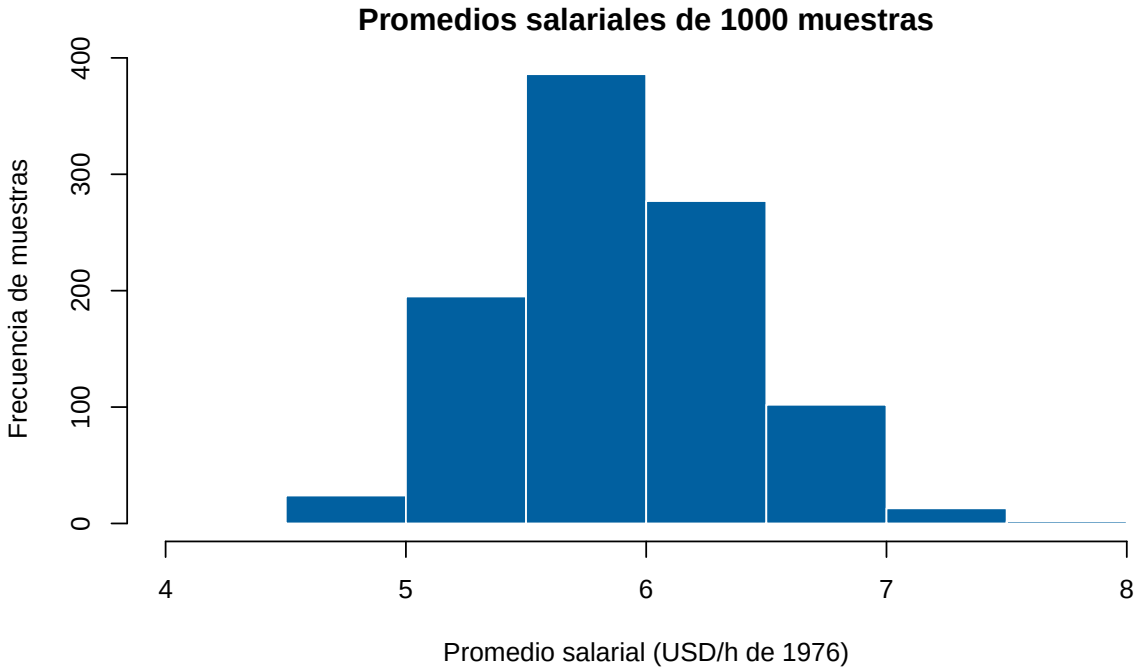
Antigüedad	Personas	Media (USD/h)	Mediana (USD/h)
0 años	163	4,531	3,750
1 a 3 años	156	5,362	4,505
4 a 8 años	102	6,908	6,000
Más de 8 años	105	7,826	6,250

Las medianas aumentan entre los grupos, aunque las distribuciones se superponen. El grupo de más de 8 años presenta una caja más amplia y se observan valores altos fuera de los bigotes en todos los grupos. Estos puntos no son automáticamente errores. La comparación describe una asociación y no prueba que aumentar la antigüedad cause por sí solo un aumento salarial.

Con `right = TRUE`, cada intervalo incluye su límite derecho: 0 queda en el primer grupo, 3 en el segundo y 8 en el tercero. Los cuatro grupos reúnen las 526 personas de la base.

7. Histograma de promedios muestrales

```
avg_wage <- numeric(1000)
set.seed(123)
for (i in 1:1000) {
  filas <- sample(1:nrow(datos), size = 50, replace = FALSE)
  avg_wage[i] <- mean(datos$wage[filas])
}
hist(avg_wage, main = "Promedios salariales de 1000 muestras",
     xlab = "Promedio salarial (USD/h de 1976)",
     ylab = "Frecuencia de muestras",
     col = "#0160a0", border = "white")
head(avg_wage)
mean(avg_wage)
sd(avg_wage)
```



Descripción de lo observado. Se observa una distribución unimodal, aproximadamente acampanada, con la mayor concentración de promedios alrededor de 5,9 USD/h y pocos valores en los extremos. La figura muestra la distribución de **1.000 promedios muestrales**, no la de 1.000 salarios individuales.

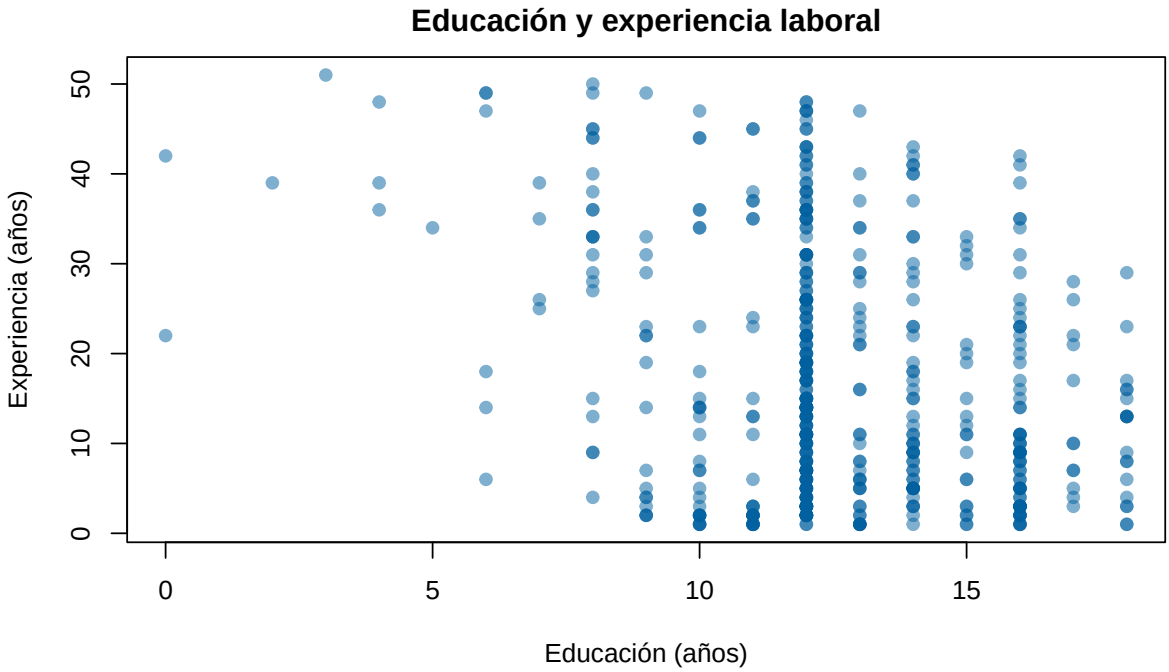
La forma aproximadamente acampanada de esta distribución es evidencia empírica compatible con el Teorema Central del Límite en este experimento.

Con `set.seed(123)`, los primeros seis promedios son **5,9300; 5,7710; 4,7260; 6,4020; 6,3214 y 5,5564**. La media de los 1.000 promedios es aproximadamente **5,8964 USD/h** y su desviación estándar es **0,5006 USD/h**.

La semilla se fija una sola vez, antes del ciclo. Dentro de cada muestra no se repiten filas; entre distintas muestras sí pueden volver a aparecer. `avg_wage[i]` guarda un promedio por posición.

8. Educación, experiencia y correlación

```
plot(datos$educ, datos$exper, pch = 19,
     col = "#0160a080", main = "Educación y experiencia laboral",
     xlab = "Educación (años)", ylab = "Experiencia (años)")
cor(datos$educ, datos$exper)
```



La correlación de Pearson es aproximadamente $r = -0{,}2995$. No se observa una relación lineal marcada entre los años de educación y la experiencia laboral. Al comparar los valores cercanos a 0 años de educación con los superiores a 15 años, se aprecia una tendencia negativa. Sin embargo, la experiencia varía mucho dentro de cada nivel educativo, lo que hace que la relación sea débil. El coeficiente se calcula con todas las observaciones, no solo con esos extremos.

Las columnas de puntos se deben a que muchas personas tienen la misma cantidad de años de educación. A un mismo nivel educativo se observan diferentes años de experiencia. El coeficiente no es cercano a -1, por lo que la relación no es una línea descendente perfecta.

Esta comparación no demuestra causalidad. La edad y otras características pueden relacionarse con ambas variables; el gráfico y la correlación no las mantienen constantes.